LESSON 14 · CHEAT SHEET

Parallel Streams & Performance

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. Parallel = **split** (Spliterator) → **run** (commonPool + caller) → **combine**. 8
2. **Sequential by default.** Parallel only for large N × Q, CPU-bound, measured.
3. **No shared mutable state.** Let `collect/toList` build results.
4. `reduce`: associative operator, **neutral** identity.
5. `toList/findFirst` keep order; `forEach/findAny` don't.
6. Never block (HTTP, JDBC) inside a parallel stream: one pool for the whole JVM.
7. Blocking I/O at scale → virtual threads 21. Timing → JMH.

Canonical snippets

1 · Starting value: add it once, outside reduce

```
opening.add(payments.parallelStream()
    .map(Payment::amount)
    .reduce(ZERO, BigDecimal::add));    // 4529.84
```

2 · Safe parallel collection, in order

```
payments.parallelStream()
    .filter(p -> p.status() == SUCCESS)
    .map(Payment::id).toList();
```

3 · Blocking I/O: virtual threads

```
try (var ex = Executors
       .newVirtualThreadPerTaskExecutor()) {
  // submit all, then get() each
}
```

Helps vs hurts

Helps	Hurts
large N × Q	small data
array, ArrayList, range	LinkedList, iterate, I/O lines
pure functions	shared mutable state
CPU-bound work	blocking I/O
unordered / cheap combine	limit, findFirst, sorted on big data
IntStream / LongStream	boxing in hot loops

Real facts (theory/HowParallelWorks)

```
ArrayList(16).trySplit()   8 + 8
LinkedList(16).trySplit()  16 + 0
commonPool parallelism     cores - 1
parallel()..sequential()   isParallel=false
worker isVirtual()         false
```

Mini diagram: split, run, combine



Real results from the course code

Call	Result
`reduce(0, a - b)` 1..10	-55 sequential; parallel differs
`reduce(10, +)` 1..10	65 sequential; parallel > 65
shared ArrayList, 100000 adds	95301 · 94292 · 100000 (3 runs)
`filter(>500).findFirst()`	P005, every run

Worked example / exercise	Result
fee report, 3 versions × 20 runs	TOTAL FEES 42.95 every time
broken → fixed float total	4829.84 → 4529.84
broken → fixed first over 500	P008 → P005

Anti-patterns → fixes

Don't	Do
`parallel()` everywhere	sequential; parallel with evidence
`forEach(list::add)`	`toList()` / `collect`
`reduce(start, op)`	`start op reduce(identity, op)`

Don't	Do
`findAny` for "the first"	`findFirst`
HTTP call in `parallelStream`	virtual threads 21
one `nanoTime` measurement	JMH, realistic data, target hardware

Lesson 14 · sticky notes

Cut out, stick on your wall, and read one every morning.

Split · run · combine

Spliterator → commonPool
→ combiner

Sequential by default

parallel only with
JMH evidence

No shared state

forEach(list::add) ×
toList() ✓

reduce rules

associative op
neutral identity

findFirst vs findAny

"the first" → findFirst
"any one" → findAny

One pool per JVM

never block inside
a parallel stream

I/O at scale [21]

virtual threads,
not parallelStream

Last call wins

parallel()..sequential()
→ all sequential